
DATA TYPES, TYPE SYSTEMS, AND TYPE CHECKING



Learning Objectives

By the end of this lecture, students will be able to:

1. Understand **what data types are and why they matter**
 2. Explain **type systems** and their classifications
 3. Differentiate **static vs dynamic type checking**
 4. Identify **type errors and type safety**
 5. Relate concepts to **real programming languages**
-

WHY DO WE NEED DATA TYPES?

- ❑ Help the compiler/interpreter understand data
 - ❑ Prevent invalid operations
 - ❑ Improve program reliability
 - ❑ Enable optimization
 - ❑ Improve readability and maintenance
-
- *Example:*
Adding two numbers vs adding a number and a string
-

What Is a Data Type?

Definition:

A **data type** defines:

- The kind of data
 - The operations allowed on it
 - The way data is stored in memory
-

Basic (Primitive) Data Types

Common primitive data types:

- Integer (int)
- Floating point (float, double)
- Character (char)
- Boolean (true/false)



Found in almost all programming languages

Derived & Composite Data Types

- Arrays
- Structures / Records
- Classes / Objects
- Strings
- Tuples



Combine multiple values into one unit

Abstract Data Types (ADT)

- Focus on **what** operations are allowed, not **how**
 - Examples:
 - Stack (push, pop)
 - Queue (enqueue, dequeue)
 - List, Map
 - 💡 ADTs improve **modularity and abstraction**
-

What Is a Type System?

Definition:

A **type system** is a set of rules that:

- Assigns types to program elements
- Enforces how types interact
-  Prevents meaningless operations

PURPOSE OF A TYPE SYSTEM

- Detect errors early
 - Enforce correctness
 - Improve documentation
 - Enable compiler optimizations
 - Increase program safety
-

TYPE ERRORS

Occurs when:

- Invalid operation on a data type
- Incompatible types used together

Example:

```
int x = "hello";
```



CLASSIFICATION OF TYPE SYSTEMS

Type systems can be classified by:

- Static vs Dynamic typing
 - Strong vs Weak typing
 - Explicit vs Implicit typing
-

Static Typing

- Type checking at **compile time**
- Errors detected early
- More verbose but safer

Languages:

- Java
 - C
 - C++
 - C#
-

Dynamic Typing

- Type checking at **runtime**
- More flexible
- Errors appear during execution

Languages:

- Python
 - JavaScript
 - Ruby
-

STATIC VS DYNAMIC TYPING (COMPARISON)

Feature	Static	Dynamic
Error Detection	Compile time	Runtime
Flexibility	Lower	Higher
Performance	Faster	Slower
Safety	Higher	Lower

STRONG TYPING

- Strict rules on type usage
- No implicit unsafe conversions

Example (Python):

"5" + 3 ❌

WEAK TYPING

- Allows implicit type conversion

Example (JavaScript):

```
"5" + 3 // "53"
```

Can cause unexpected behavior

EXPLICIT VS IMPLICIT TYPING

Explicit Typing

- Programmer declares type

```
int x =10;
```

Implicit Typing

- Compiler infers type
- `x = 10`

WHAT IS TYPE CHECKING?

Type Checking:

- Process of verifying type correctness
- Ensures operations are applied to valid types

Compile-Time Type Checking

- ❖ Done before execution
- ❖ Faster error detection
- ❖ Used in statically typed languages



Reduces runtime failures

RUNTIME TYPE CHECKING

- Performed during execution
- Common in dynamically typed languages
- More flexible, but risky

Type Safety

A language is **type-safe** if:

- It prevents type errors during execution

Type-safe languages:

- Java
- Python

Less type-safe:

- C (pointer misuse)
-

Real-World Examples

- Banking systems → **Static typing for safety**
 - Rapid prototyping → **Dynamic typing for speed**
 - Embedded systems → **Strong static typing**
-

END
